Bootstrap Recovery Liveness under Validator Loss, with Preserved Consensus-Finality Safety

Lux Industries — Consensus Group
Companion proof to LP-304

2026-06-27

Abstract

We formalise the separation, introduced in LP-304, of the *bootstrap trust* threshold from the *consensus finality* threshold. We model a primary network of weighted validators a subset of which may be offline, including the very nodes attempting to recover. We prove (i) the prior design, which named the bootstrap frontier by a quorum over $> \frac{2}{3}$ of *current total stake*, has a reachable deadlock state in which no node can bootstrap; (ii) the LP-304 policy, naming the frontier from an authenticated configured-beacon supermajority with a response floor, is live whenever an honest configured-beacon supermajority is reachable, even with more than $\frac{1}{3}$ of validators offline; and (iii) the two-threshold decoupling preserves BFT finality safety: a block is accepted only under $> \frac{2}{3}$ of *current* validator stake, and the bootstrap anchor never finalises. The third result is the weak-subjectivity soundness condition ("C1") under which the liveness gain costs no safety.

1 Model

Definition 1 (Validator set and stake)**.** *Let $V = \{v_1, \ldots, v_n\}$ be the current primary-network validators with stake weights $w : V \to \mathbb{Z}_{>0}$ and total $W = \sum_{v \in V} w(v)$. The two-thirds floor is $\mathsf{F}(W) = \lfloor 2W/3 \rfloor$; a stake mass s* clears *the floor iff $s > \mathsf{F}(W)$.*

Definition 2 (Reachability)**.** *At a point in time a subset $R \subseteq V$ is* reachable *(connected and responding). A node u attempting to recover is itself offline-for-recovery: $u \notin R$. Write $W_R = \sum_{v \in R} w(v)$ for the reachable stake.*

Definition 3 (Configured beacon anchor)**.** *Independent of V, each node holds a* configured beacon set *B resolved, in priority order, from (1) operator configuration, (2) a trusted finalized checkpoint, or (3) genesis. B is fixed at the node and **not** derived from peer self-report. Let $B_R = B \cap R$ be the reachable beacons and $\mathsf{honest}(B_R) \subseteq B_R$ the honest ones.*

Definition 4 (Bootstrap policy, LP-304)**.** *A $\mathsf{BootstrapPolicy}$ is $(B,\ \rho,\ m,\ \mathsf{cp})$ with agreement ratio $\rho \in \{\frac{2}{3}, \frac{3}{4}\}$, response floor $m \in \mathbb{Z}_{\geq 2}$, and optional checkpoint cp. On authenticated replies from responders B_R, it **accepts** a frontier block b iff*

$$|B_R| \ \geq\ m \quad \textit{and} \quad w(\{\, v \in B_R : b \preceq \mathsf{tip}(v) \,\}) \ >\ \rho \cdot W_{B_R},$$

where $b \preceq \mathsf{tip}(v)$ means b is the reported tip of v or an ancestor of it (ancestor-tolerant naming, LP-304 §algorithm), and $W_{B_R} = \sum_{v \in B_R} w(v)$ is the responding *beacon weight; or cp explicitly authorises b. The denominator is the responding beacons, never W.*

2 Unsatisfiability of the prior (current-stake) bootstrap threshold

Definition 5 (Prior threshold)**.** *The prior design names a frontier only if the stake of validators agreeing on it clears* $\mathsf{F}(W)$ *over the* current total W*.*

Theorem 1 (Mass-recovery deadlock)**.** *There is a reachable state in which the prior threshold can never be met, hence no offline node bootstraps, even though the network is otherwise correct.*

Proof. Take $n = 5$, $w \equiv c$, so $W = 5c$ and $\mathsf{F}(W) = \lfloor 10c/3 \rfloor = 3c$ (for c a multiple of 3, and $3c < 10c/3$ so the strict floor is $3c$). Let two validators be offline for recovery, $R = \{v_1, v_3, v_4\}$, $W_R = 3c$. The maximum stake that can agree on any frontier is $W_R = 3c$, and $3c \not> 3c$. The floor is never cleared, independent of agreement quality or ancestor tolerance. The two offline validators contribute the missing $2c$; but they are exactly the nodes whose recovery is blocked. The blocked set and the missing stake coincide, so the state is a fixed point: recovery requires their stake, their stake requires their recovery. □ □

Remark 1. *Theorem 1 is not an eclipse or a fault: R is honest and reachable. It is a threshold-model error — the* finality *denominator W used for a* liveness *(bootstrap) decision. This is precisely the mainnet state observed for* `luxd-2` *(LP-304, §empirical): $W = 2.5 \times 10^{18}$, $\mathsf{F} = 1.\overline{6} \times 10^{18}$, $W_R = 1.5 \times 10^{18} = 60\% < 66.\overline{6}\%$.*

3 Liveness of the LP-304 policy

Theorem 2 (Recovery liveness under validator loss)**.** *Let the policy be $(B, \rho, m, \cdot)$ with $\rho \leq \frac{3}{4}$. Suppose the reachable honest beacons satisfy $|B_R| \geq m$ and $w(\mathsf{honest}(B_R)) > \rho \cdot W_{B_R}$, and the honest beacons agree (tip-or-ancestor) on a common block $b^\star$. Then the policy accepts a frontier $b \succeq$ the honest-agreed height and the node syncs to it — even if $W_R \leq \mathsf{F}(W)$, i.e. more than $\frac{1}{3}$ of* total *validators are offline.*

Proof. The honest beacons all have $b^\star \preceq \mathsf{tip}(\cdot)$, so the agreeing weight on $b^\star$ is at least $w(\mathsf{honest}(B_R)) > \rho W_{B_R}$, clearing the policy's *responding-beacon* threshold; and $|B_R| \geq m$ clears the floor. By Definition 4 the policy accepts $b^\star$ (or a higher block on the same chain with the same property; ancestor tolerance selects the highest such). Acceptance depends only on W_{B_R} and $|B_R|$, never on W; the offline validators' absence from B_R does not enter. Hence acceptance holds for any amount of offline *total* stake, including $W_R \leq \mathsf{F}(W)$, as long as the honest reachable-beacon supermajority condition holds. For the $n = 5$, $R = \{v_1, v_3, v_4\}$ instance with $B = V$, $m = 3$, $\rho = \frac{2}{3}$: $|B_R| = 3 \geq 3$ and (all three honest) the agreeing weight $3c > \frac{2}{3} \cdot 3c = 2c$, so $b^\star = 1082796$ is accepted — the case Theorem 1 deadlocks. □ □

Proposition 3 (Anti partition-capture)**.** *With $m \geq 2$ and the strict ratio ρ, an attacker who partitions the node to $|B_R| < m$ reachable beacons cannot make the policy accept any frontier, regardless of their unanimity.*

Proof. Acceptance requires $|B_R| \geq m$. If $|B_R| < m$ the first conjunct of Definition 4 fails outright; unanimity (ρW_{B_R} trivially cleared) is irrelevant. Only an explicit operator checkpoint cp can authorise a frontier below the floor, by deliberate operator action. □ □

4 Finality safety is preserved (weak-subjectivity soundness, "C1")

Definition 6 (Finality)**.** *A block is* final *iff it carries a verified α-of-K quorum certificate whose signing weight clears* $\mathsf{F}(W)$ *over the* current *validator stake (LP-045/142).* $\mathsf{HasFinality}(s, W) \equiv s > \mathsf{F}(W)$.

Theorem 4 (Decoupling preserves safety)**.** *The LP-304 separation does not change the finality predicate: a block is accepted as final iff* HasFinality *holds on current stake. The bootstrap policy never finalises a block, and no frontier it names is treated as final.*

Proof. BootstrapTrust.AcceptsFrontier returns a value of type Frontier (a sync target), never a certificate, and the descent re-executes and re-verifies every block before the node re-enters consensus (Invariant 3); after re-entry, acceptance is dispatched solely through ConsensusQuorum.HasFinality on the current stake set, syntactically distinct from BootstrapTrust. The two interfaces share no implementation: the policy's denominator is responding-beacon weight W_{B_R}, the finality denominator is current stake W. Therefore for any block b marked final there is a certificate with weight $> \mathsf{F}(W)$; the bootstrap path produces no such certificate and cannot mark b final. □ □

Theorem 5 (Weak-subjective soundness of the named frontier)**.** *Under the assumption that the configured beacon set contains less than a ρ-fraction of Byzantine responding weight, the policy never names a forged (non-canonical) frontier.*

Proof. A forged block f is on $\mathsf{tip}(v)$'s chain only for Byzantine v (an honest beacon reports its real accepted chain, and the ancestor relation $f \preceq \mathsf{tip}(v)$ is content-addressed: f's id is the hash of its bytes, and the descent follows verified parent links, so a substituted parent yields a different id and is unreachable — LP-304 §rationale). Hence the agreeing weight on f is at most the Byzantine responding weight $< \rho\, W_{B_R}$, which does not clear the policy threshold; f is never named (tests D, E). The canonical block, backed by the honest $> \rho$ supermajority, is named instead. An attacker cannot enlarge the Byzantine fraction by injecting beacons, since B is resolved from configuration/checkpoint/genesis and a non-configured peer has zero weight (Invariant 1, test D). □ □

5 Caught-up by acceptance, not store presence

The policy of §4 names the correct frontier $b^\star$, but a node only *converges* if it then *recognises* that it has reached $b^\star$ and otherwise advances toward it. Two further degrees of freedom in that recognition — which set of blocks counts as "reached," and which height the loop reads to detect advancement — admit a non-converging fixed point that named-frontier correctness alone does not exclude. This was the final live-mainnet defect (LP-304, layer 8).

Definition 7 (Held vs. accepted)**.** *A recovering node u has two monotone heights: its* accepted height $\mathsf{acc}(u)$ *(the top of its consensus-accepted chain) and its* held height $\mathsf{held}(u) \geq \mathsf{acc}(u)$ *(the highest block present in its block store). Block gossip may raise* $\mathsf{held}(u)$ *above* $\mathsf{acc}(u)$*: the node* has *blocks it has not* accepted*. Let $N^\star$ be the named frontier height (Thm 2), with $N^\star > \mathsf{acc}(u)$.*

Definition 8 (Caught-up predicates)**.** *The* store-presence *predicate is* $\mathsf{CU}_{\mathsf{Has}}(u) \equiv \mathsf{held}(u) \geq N^\star$ *(the named tip is present in the store). The* acceptance *predicate is* $\mathsf{CU}_{\mathsf{Acc}}(u) \equiv \mathsf{acc}(u) \geq N^\star$ *(the named tip is on the accepted chain). When caught-up is false, the loop runs one* cert-gated accept step*: it accepts the held block at height* $\mathsf{acc}(u)+1$ *iff that block carries a verifying α-of-K certificate, raising* $\mathsf{acc}(u)$ *by one; with no caught-up signal it instead concludes and goes live.*

Theorem 6 (Store-presence caught-up is a non-converging fixed point)**.** *There is a reachable state with* $\mathsf{held}(u) \geq N^\star > \mathsf{acc}(u)$ *in which* $\mathsf{CU}_{\mathsf{Has}}(u)$ *holds, so the loop concludes "caught up" and runs no accept step, yet u is not at the frontier. The state is a fixed point:* $\mathsf{acc}(u)$ *never advances, and every restart re-enters it.*

Proof. Let u have accepted to $\mathsf{acc}(u) = h_0$ and received by gossip every block up to $N^\star = h_0 + k$, so $\mathsf{held}(u) = h_0 + k \geq N^\star$. Then $\mathsf{CU}_{\mathsf{Has}}(u) = (\mathsf{held}(u) \geq N^\star) = \text{true}$, so the loop takes the caught-up branch and performs no accept step; $\mathsf{acc}(u)$ stays $h_0 < N^\star$. Nothing in the branch mutates held or acc, so the state is fixed. On restart the store still holds the blocks, $\mathsf{CU}_{\mathsf{Has}}$ is again true, and the same branch is taken — the defect is self-reinforcing. The node serves a stale accepted chain while believing it is current. □ □

Remark 2. *This is exactly the observed mainnet state:* `luxd-2` *logged* `NAMED namedHeight=1082796 held=true` *with* acc = 1082780 *— it* held 1082781..1082796 *by gossip but had* accepted *only to* 1082780*, and the store-presence predicate (resolving "held" through the VM's* `GetBlock`*, which returns stored* or *accepted blocks alike) reported caught-up at the stale height (LP-304, layer 8).*

Theorem 7 (Acceptance caught-up converges, safely)**.** *Using* $\mathsf{CU}_{\mathsf{Acc}}$*, a node with* $\mathsf{held}(u) \geq N^\star > \mathsf{acc}(u)$ *and a valid certificate for each held block on the named chain reaches* $\mathsf{acc}(u) = N^\star$ *in exactly* $N^\star - \mathsf{acc}(u)$ *accept steps, and no block is accepted without its certificate.*

Proof. While $\mathsf{acc}(u) < N^\star$, $\mathsf{CU}_{\mathsf{Acc}}(u)$ is false, so the loop runs an accept step. The block at height $\mathsf{acc}(u) + 1$ is present (since $\mathsf{held}(u) \geq N^\star > \mathsf{acc}(u) + 1$ until the last step) and carries a verifying certificate by hypothesis, so the cert-gated guard accepts it and $\mathsf{acc}(u)$ increases by exactly one. The accept step strictly increases a quantity bounded above by $N^\star$, so after $N^\star - \mathsf{acc}_0$ steps $\mathsf{acc}(u) = N^\star$ and $\mathsf{CU}_{\mathsf{Acc}}$ becomes true. Safety: the guard accepts a block *only* on a verifying α-of-K certificate, so a stored block lacking one (a forged or off-chain block in the store) is never accepted merely for being present — store presence confers no acceptance. □ □

Lemma 8 (Convergence recognition must read the advancing ledger)**.** *Suppose the accept step advances the in-process finalised height* $\mathsf{acc}(u)$*, but the loop evaluates caught-up against a cached value* $\widehat{\mathsf{acc}}(u)$ *that is written once at initialisation and never refreshed. Then even under* $\mathsf{CU}_{\mathsf{Acc}}$ *the node fails to recognise convergence: it advances on disk yet loops forever. Reading* $\mathsf{acc}(u)$ *from the finalised ledger restores Thm 7.*

Proof. $\mathsf{CU}_{\mathsf{Acc}}$ evaluated on the frozen cache is $\widehat{\mathsf{acc}}(u) \geq N^\star$, with $\widehat{\mathsf{acc}}(u) = \mathsf{acc}_0 < N^\star$ constant. It is false at every step regardless of the real $\mathsf{acc}(u)$; the loop keeps running accept steps (or, past the top, finds nothing to accept and stalls) and never reports caught-up — an advance-then-stall. Substituting the live finalised height $\mathsf{acc}(u)$ for $\widehat{\mathsf{acc}}(u)$ makes the predicate the one of Thm 7, which converges. □ □

Remark 3. *The frozen cache is the plugin (zap) RPC client's* `LastAccepted`*, written at initialisation and not on* `Accept`*; the live source is the consensus finalised ledger that the per-height accept guard already maintains. Driving recognition off the ledger (and refreshing the client cache on* `Accept`*, since the stale value was also served to peers) is the layer-8 fix. The unit mocks hid both Thm 6 and Lemma 8 because the mock VM advanced* `LastAccepted` *on accept and resolved "held" as "accepted" — it was* higher-fidelity *than the real host in exactly the two ways that mask the bug, so a green suite did not imply convergence.*

6 Conclusion

Corollary 9. *Separating* BootstrapTrust *from* ConsensusQuorum *converts an unsatisfiable recovery (Thm 1) into a live one under realistic validator loss (Thm 2, Prop 3) at no cost to finality safety (Thm 4) and no new forgery surface under the standard weak-subjectivity assumption (Thm 5). Convergence is then* recognised *correctly only when caught-up is defined by acceptance, not store presence (Thm 6, Thm 7), and is read from the advancing finalised ledger, not a frozen cache (Lemma 8); both are cert-gated, so safety is preserved. Validators may freely leave and rejoin: a rejoining node bootstraps from whichever configured beacons are reachable and agreeing, accepts the gap under quorum certificates, and finalises on the current stake set. The liveness threshold and the safety threshold are distinct objects with distinct denominators, and "reached the frontier" is the accepted chain, never the store — none can be re-conflated.*